Aptos Securitize

Security Assessment

November 5th, 2024 — Prepared by OtterSec

Andreas Mantzoutas andreas@osec.io

Bartłomiej Wierzbiński dark@osec.io

Robert Chen r@osec.io

Table of Contents

Executive Summary **3**

Overview 3

Key Findings 3

Scope **4**

Findings **5**

Vulnerabilities **6**

OS-ASC-ADV-00 | Logic Flaw in Time Check 9

OS-ASC-ADV-01 | Absence of State Variable Update on Wallet Removal 10

OS-ASC-ADV-02 | Incorrect Issuance Value Recording 12

OS-ASC-ADV-03 | Incorrect Lock Removal Logic 13

OS-ASC-ADV-04 | Artificial Reduction of Investor Counts 14

OS-ASC-ADV-05 | Disabling Withdrawals by Withdrawing Zero-Value FA 16

OS-ASC-ADV-06 | Flaw in Full Transfer Checks 17

OS-ASC-ADV-07 | Discrepancies in Updating Investor Count 18

OS-ASC-ADV-08 | Utilization of Proper Assertions for Wallet Creation 19

OS-ASC-ADV-09 | Abort Due to Underflow in Difference Calculation 21

OS-ASC-ADV-10 | Wallet Balance Misverification 22

OS-ASC-ADV-11 | Missing Bound Check on Lock Removal 24

OS-ASC-ADV-12 | Failure to Clear Investor Attributes After Removal 25

OS-ASC-ADV-13 | Investor Limit Calculation Error 26

OS-ASC-ADV-14 | Incongruities in Role Management Checks 27

OS-ASC-ADV-15 | Utilization of Empty Investor ID 28

© 2024 Otter Audits LLC. All Rights Reserved.

OS-ASC-ADV-16 | Missing Last Modifier Update 29

General Findings **30**

OS-ASC-SUG-00 | Missing Approval Removal Function 31

OS-ASC-SUG-01 | Missing Validation Logic 32

OS-ASC-SUG-02 | Additional Safety Checks 33

OS-ASC-SUG-03 | Code Refactoring 34

Appendices

Vulnerability Rating Scale **36**

Procedure **37**

© 2024 Otter Audits LLC. All Rights Reserved.

01 — Executive Summary

Overview

Aptos engaged OtterSec to assess the `securitize` program. This assessment was conducted between October 14th and October 29th, 2024. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 21 findings throughout this audit engagement.

In particular, we identified several vulnerabilities related to the utilization of zero-amount fungible assets, including the possibility of users with zero balances repeatedly depositing and withdrawing zero-value assets, artificially decreasing the total investor count, preventing the proper removal of inactive investors (OS-ASC-ADV-04), and causing the withdrawal count to be in a state that blocks further withdrawals when zero-value fungible assets are withdrawn and destroyed without depositing (OS-ASC-ADV-05). We also highlighted several inconsistencies in the locking logic, concerning the incorrect deletion of the last lock record, resulting in inaccuracies in tracking transferable tokens (OS-ASC-ADV-03), and the lack of proper validation and failure to fully remove lock records, allowing the same lock to be removed multiple times (OS-ASC-ADV-11).

Furthermore, removing an investor does not clear the attributes associated with the investor when they are removed from the registry (OS-ASC-ADV-12), and a flaw in the full transfer checks incorrectly blocks all transfers from the US region to the platform wallet (OS-ASC-ADV-06).

Additionally, we identified numerous discrepancies in the role management verifications, which are flawed in multiple functions, resulting in improper access control and inconsistency between the actual implementation and the documentation (OS-ASC-ADV-14). There are also multiple instances where the investor count is not correctly updated, particularly during burn events and when adjusting for a change in an investor's country (OS-ASC-ADV-07).

We also recommended adding a function to revoke any given spender's permission by deleting their entry from the allowances list altogether (OS-ASC-SUG-00) and advised incorporating additional checks within the codebase for improved robustness and security (OS-ASC-SUG-01). We further suggested modifying the codebase for improved functionality, efficiency, and maintainability (OS-ASC-SUG-03).

© 2024 Otter Audits LLC. All Rights Reserved.

02 — Scope

The source code was delivered to us in a Git repository at https://github.com/aptos-labs/aptos-securitize. This audit was performed against commit bd5269f.

A brief description of the program is as follows:

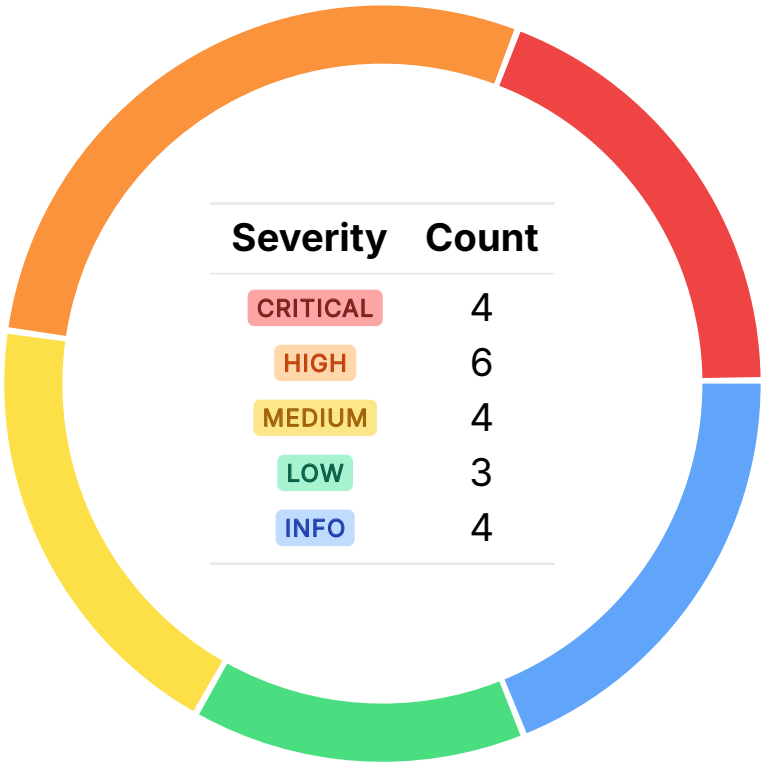
Name	Description
securitize	It consists of a set of smart contracts to manage regulated digital securities.

© 2024 Otter Audits LLC. All Rights Reserved.

03 — Findings

Overall, we reported 21 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	4
HIGH	6
MEDIUM	4
LOW	3
INFO	4

© 2024 Otter Audits LLC. All Rights Reserved.

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in Appendix A.

ID	Severity	Status	Description
OS-ASC-ADV-00	CRITICAL	RESOLVED	`is_block_flowback_end_time_ok` contains a logic flaw where it always returns true if `block_flowback_end_time` is zero, preventing any subsequent checks against the current timestamp.
OS-ASC-ADV-01	CRITICAL	RESOLVED	`remove_wallet` fails to update the `investor.wallets` after removing a wallet, resulting in inaccurate balance calculations in `investor_wallet_balance_total`.
OS-ASC-ADV-02	CRITICAL	RESOLVED	`create_issuance_information` incorrectly sets issuance values to zero instead of `utilizing` the `_value` parameter provided.
OS-ASC-ADV-03	CRITICAL	RESOLVED	`remove_lock_record_for_investor` incorrectly deletes the last lock record instead of the specified lock, resulting in inaccuracies in tracking transferable tokens.
OS-ASC-ADV-04	HIGH	RESOLVED	Users with zero balances may repeatedly deposit and withdraw zero-value assets, artificially decreasing the total investor count, and preventing proper removal of inactive investors.
OS-ASC-ADV-05	HIGH	RESOLVED	When zero-value FAs are withdrawn and destroyed without being deposited, the `WithdrawCount` is left in a state that blocks further withdrawals by violating the `assert_withdraw_count` condition.

© 2024 Otter Audits LLC. All Rights Reserved.

OS-ASC-ADV-06	HIGH	RESOLVED	A flaw in the full transfer checks incorrectly blocks all transfers from the US region to the platform wallet if `get_force_full_transfer` is set, without verifying whether the transfer amount matches the investor's balance.
OS-ASC-ADV-07	HIGH	RESOLVED	There are multiple instances where the investor count is not correctly updated, particularly during burn events and when adjusting for a change in an investor's country.
OS-ASC-ADV-08	HIGH	RESOLVED	It is possible to prevent registration of wallet addresses due to the utilization of `create_primary_store` instead of `ensure_primary_store_exists` in `add_wallet`.
OS-ASC-ADV-09	HIGH	RESOLVED	A potential underflow scenario exists if `lock_time` is less than `time`.
OS-ASC-ADV-10	MEDIUM	RESOLVED	`check_wallets_for_list` incorrectly verifies wallet balances based on investor balances, allowing the addition of zero-balance wallets to the system.
OS-ASC-ADV-11	MEDIUM	RESOLVED	The lock removal process lacks proper validation and does not fully remove lock records, allowing the same lock to be removed multiple times and resulting in inaccurate lock information.
OS-ASC-ADV-12	MEDIUM	RESOLVED	`remove_investor` does not clear the attributes associated with an investor when they are removed from the registry.
OS-ASC-ADV-13	MEDIUM	RESOLVED	`max_us_investors_percentage` is not considered when calculating the limit for U.S. investors, resulting in potential inaccuracies.

© 2024 Otter Audits LLC. All Rights Reserved.

OS-ASC-ADV-14	LOW	RESOLVED	The role management verifications are flawed in multiple functions, resulting in improper access control and inconsistencies between the actual implementation and the documentation.
OS-ASC-ADV-15	LOW	RESOLVED	`register_investor` lacks a check to prevent empty string IDs, which may result in significant logical errors in the contract.
OS-ASC-ADV-16	LOW	RESOLVED	`set_attribute` currently does not update the `last_updated_by` field in the investor's record, which may result in outdated tracking of who last modified the investor's attributes.

© 2024 Otter Audits LLC. All Rights Reserved.

Logic Flaw in Time Check CRITICAL OS-ASC-ADV-00

Description

`is_block_flowback_end_time_ok` is checking the condition incorrectly. If `block_flowback_end_time` is zero, the first condition (`block_flowback_end_time != 0`) evaluates to false, and the function will never abort, as the second condition (`timestamp` < `block_flowback_end_time`) will also evaluate to false because `timestamp::now_seconds` will always be greater than or equal to zero.

>_ *move/sources/compliance_service.move* RUST

```rust
/// Checks if the block flowback end time has been reached.
fun is_block_flowback_end_time_ok(block_flowback_end_time: u256): bool {
    block_flowback_end_time != 0
        || (timestamp::now_seconds() as u256) < block_flowback_end_time
}
```

In the case where `block_flowback_end_time` is not zero, `is_block_flowback_end_time_ok` will always return true, resulting in the function aborting. As a result, it will not check the `timestamp::now_seconds() as u256) < block_flowback_end_time` condition.

Remediation

Return true when `block_flowback_end_time` is 0, allowing all operations, and if `block_flowback_end_time` is greater than zero, evaluate whether the current timestamp is less than the flowback end time. If it is, it indicates the flowback period is still active, and the function should return true. If not, return false, indicating that the flowback period has ended.

Patch

Resolved in 4e2290c.

© 2024 Otter Audits LLC. All Rights Reserved.

Absence of State Variable Update on Wallet Removal CRITICAL OS-ASC-ADV-01

Description

In `registry_service::remove_wallet`, the failure to remove `wallet_addr` from the `investor.wallets` structure introduces a significant vulnerability. This results in inaccurate calculations in `investor_wallet_balance_total`, which, in turn, impacts various other functions in the project that depend on accurate wallet balances. `investor_wallet_balance_total` calculates the total balance across all wallets associated with an investor by iterating through `investor.wallets`.

>_ *move/sources/registry_service.move* RUST

```rust
public entry fun remove_wallet(
    authorizer: &signer, wallet_addr: address, id: String
) acquires InvestorInfo {
    [...]
    let investor_wallets_mut = &mut investor_info_mut().investors_wallets;
    smart_table::remove(investor_wallets_mut, wallet_addr);
    let investor = smart_table::borrow_mut(&mut investor_info_mut().investors, id);
    investor.wallet_count = investor.wallet_count - 1;
    event::emit(
        DSRegistryServiceWalletRemoved {
            wallet: wallet_addr,
            investorId: id,
            sender: authorizer_address
        }
    );
}
```

`remove_wallet` does not remove the `wallet_addr` from `investor.wallets`, it remains in this list even though it is reducing the `wallet_count` by one. `investor_wallet_balance_total` iterates over investor wallets until `wallet_count` is reached. However, since `wallet_count` was incorrectly reduced, the iteration may stop prematurely.

>_ *move/sources/registry_service.move* RUST

```rust
public fun investor_wallet_balance_total(investor_id: String): u64 acquires InvestorInfo {
    [...]
    for (i in 0..investor.wallet_count) {
        let wallet = *vector::borrow(&investor.wallets, i);
        let pfs = primary_fungible_store::primary_store_inlined(wallet, asset);
        balance = balance + fungible_asset::balance(pfs);
    };
    balance
}
```

© 2024 Otter Audits LLC. All Rights Reserved.

Remediation

Ensure that the `wallet_addr` is also removed from `investor.wallets` in `remove_wallet`.

Patch

Resolved in 06538b8 and 689014d.

© 2024 Otter Audits LLC. All Rights Reserved.

Incorrect Issuance Value Recording CRITICAL OS-ASC-ADV-02

Description

`compliance_service::create_issuance_information` explicitly sets `value` to zero rather than utilizing the `_value` parameter, which implies that all recorded `issuances` will have a value of zero instead of the actual issuance amount. Because every issuance is recorded with a value of zero, the issuance information stored in `issuances_values` will not accurately represent the actual amounts.

>_ *move/sources/compliance_service.move* RUST

```rust
fun create_issuance_information(
    investor: String, _value: u256, issuance_time: u256
): bool acquires ComplianceData {
    [...]
    // Update the issuances value with respect to the investor and current issuancescount.
    let issuances_values = &mut compliance_data_mut.issuances_values;
    let value = 0;
    [...]
}
```

Remediation

Utilize the `_value` parameter so that each issuance record accurately captures the intended amount passed to the function.

Patch

Resolved in 9f5d9b2.

© 2024 Otter Audits LLC. All Rights Reserved.

Incorrect Lock Removal Logic CRITICAL OS-ASC-ADV-03

Description

`remove_lock_record_for_investor` in `lock_manager` is intended to swap the lock record at `lock_index` with the last lock in the investor's lock list, then reduce the count of locks by one. However, the implementation incorrectly overwrites the lock at `lock_index` with itself, which implies that the last lock is removed instead of the one at `lock_index`.

>_ *move/sources/lock_manager.move* RUST

```rust
public entry fun remove_lock_record_for_investor(
    authorizer: &signer, investor: String, lock_index: u64
) acquires LockInfo {
    let lock_info = lock_info_mut();
    let investor_locks = &lock_info.investor_locks;
    assert!(smart_table::contains(investor_locks, investor), EINVESTOR_NOT_LOCKED);
    let investor_locks_inner = smart_table::borrow(investor_locks, investor);
    assert!(
        smart_table::contains(investor_locks_inner, lock_index), EINVESTOR_NOT_LOCKED
    );
    let lock = smart_table::borrow(investor_locks_inner, lock_index);
    [...]
    smart_table::upsert(investor_locks_count, investor, last_lock_number);
    set_lock_info_impl(
        investor,
        lock_index,
        lock.value,
        lock.reason_code,
        lock.reason_string,
        lock.release_time,
        investor_locks_mut
    );
}
```

Remediation

Check if `total_investor_count` is less than the investor's lock count, and remove the last lock.

Patch

Resolved in 06538b8.

© 2024 Otter Audits LLC. All Rights Reserved.

Artificial Reduction of Investor Counts HIGH OS-ASC-ADV-04

Description

There is an inconsistency in the way the system allows users to create zero-value fungible assets (FA) through `fungible_asset::zero` and deposit them via `dispatchable_fungible_asset::deposit`. A user with a zero FA balance may withdraw a zero-value FA and then deposit it, setting `WithdrawCount.amount` to zero.

>_ aptos-move/framework/aptos-framework/sources/fungible_asset.move RUST

```rust
/// Create a fungible asset with zero amount.
/// This can be useful when starting a series of computations where the initial value is 0.
public fun zero<T: key>(metadata: Object<T>): FungibleAsset {
    FungibleAsset {
        metadata: object::convert(metadata),
        amount: 0,
    }
}
```

Consequently, the user may repeatedly create zero-value FAs (via `fungible_asset::zero`) and deposit them (via `dispatchable_fungible_asset::deposit`), which calls `record_transfer`. Within `record_transfer`, if an investor has a balance of zero after a transfer, it reduces the investor count by calling `adjust_total_investors_counts(from, false)`, which decreases the total number of investors in the system.

>_ move/sources/compliance _service.move RUST

```rust
/// Records the transfer between two investors.
/// Adjusts the total investors count based on balances.
fun record_transfer(
    from: address,
    to: address,
    value: u256,
): bool acquires ComplianceData {
    if (compare_investor_balance(to, value, 0)) {
        adjust_total_investors_counts(to, true);
    };
    if (balance_of_investor(from) == 0) {
        adjust_total_investors_counts(from, false);
    };
    true
}
```

© 2024 Otter Audits LLC. All Rights Reserved.

This logic assumes that a zero balance indicates that the investor no longer holds any assets in the system; thus, they should no longer be considered active investors. Consequently, it is possible to artificially reduce the investor count, which may prevent the proper removal of investors.

Remediation

Ensure that the investor is not depositing a zero amount.

Patch

Resolved in 9f5d9b2.

© 2024 Otter Audits LLC. All Rights Reserved.

Disabling Withdrawals by Withdrawing Zero-Value FA HIGH OS-ASC-ADV-05

Description

The `ds_token` module relies on an invariant that withdrawals must be coupled with deposits. The system tracks the number of active withdrawals utilizing the `WithdrawCount` resource, ensuring that multiple withdrawals will not coincide (as enforced by `assert_withdraw_count`). However, via `dispatchable_fungible_asset::withdraw`, a user may withdraw a zero-value fungible asset (FA). Since this FA has a value of zero, it does not represent any meaningful asset transfer. Still, the `WithdrawCount` is incremented to reflect that a withdrawal has occurred.

>_ *move/sources/ds_token.move* RUST

```rust
public fun withdraw<T: key>(
    store: Object<T>, amount: u64, transfer_ref: &TransferRef
): FungibleAsset acquires State, WithdrawCount {
    assert_not_paused();
    assert_is_primary_store(store);
    assert_withdraw_count();
    let from_address = object::owner(store);
    compliance_service::validate_withdraw(from_address, (amount as u256), metadata());
    increment_withdraw_count_and_amount(from_address, amount);
    fungible_asset::withdraw_with_ref(transfer_ref, store, amount)
}
```

Thus, the system will now believe that a withdrawal is still in progress, even though the zero-value FA was destroyed. As a result, future withdrawal attempts will fail because of the `assert_withdraw_count` check.

Remediation

Ensure that the investor is not withdrawing a zero amount.

Patch

Resolved in 06538b8.

© 2024 Otter Audits LLC. All Rights Reserved.

Flaw in Full Transfer Checks HIGH OS-ASC-ADV-06

Description

In `compliance_service::pre_deposit_check_regulated`, the `get_force_full_transfer` condition checks if full transfers are enforced when the transfer originates from the US. If this condition is true, the function immediately aborts the transfer with the error code `EONLY_FULL_TRANSFER`. If `get_force_full_transfer` is enabled and the transfer originates from the US, any transfer to the platform wallet is rejected, regardless of whether the transfer satisfies the required full transfer conditions. As a result, valid transactions may be blocked.

>_ *move/sources/compliance_service.move* RUST

```rust
fun pre_deposit_check_regulated(
    withdraw_from: &mut WithdrawFrom,
    to: address,
    value: u256,
    paused: bool,
): (u256, u64) acquires ComplianceData, ComplianceConfig, TransferRestrictions,TokenLimits,
    ↪  TimeConstraints {
    [...]
    if (is_platform_wallet) {
        if ((get_force_full_transfer()
            && from_region == US)
            || (
                get_world_wide_force_full_transfer()
                    && (from_investor_balance as u256) > value
            )) {
            abort EONLY_FULL_TRANSFER
        };
        return (0, VALID)
    };
    [...]
}
```

Remediation

Check whether the full transfer is forced by confirming if `get_force_full_transfer` and `from_region == US` are true, or if `get_world_wide_force_full_transfer` is set. If so, verify that the transfer is made in full (`from_investor_balance as u256 > value`).

Patch

Resolved in #61.

© 2024 Otter Audits LLC. All Rights Reserved.

Discrepancies in Updating Investor Count HIGH OS-ASC-ADV-07

Description

In the existing implementation of `compliance_service::record_burn` (shown below), the function checks if the investor's balance equals the value to be burned (`balance_who == (value as u64)`). However, it does not explicitly check if `value` is greater than zero before making this comparison or before adjusting the investor count. If `value` is zero, the condition will still evaluate to true, which will decrease the total investor count incorrectly, even though the investor was already inactive (with a zero balance) and should not have been counted.

>_ *move/sources/compliance_service.move* RUST

```rust
fun record_burn(who: address, value: u256): bool acquires ComplianceData {
    let balance_who = balance_of_investor(who);
    if (balance_who == (value as u64)) {
        adjust_total_investors_counts(who, false);
    };
    true
}
```

Additionally, `registry_service::adjust_investor_counts_after_country_change` currently only increases the investor count for the new country but does not decrease it for the previous country. If an investor changes their country, compliance counts for the old country need to be updated to reflect that the investor no longer belongs to it.

Remediation

Check if the burn value is non-zero before adjusting the investor count in `record_burn`, and ensure that `adjust_investor_counts_after_country_change` decreases the count for the previous country.

Patch

Resolved in 9f5d9b2.

© 2024 Otter Audits LLC. All Rights Reserved.

Utilization of Proper Assertions for Wallet Creation HIGH OS-ASC-ADV-08

Description

`registry_service::add_wallet` directly utilizes `create_primary_store` to create a new primary store for a specified `wallet_addr`. However, this function will fail if the address already has a primary store. Thus on calling `add_wallet` or `add_wallet_by_investor`, if the wallet already has a primary store it will not be added and the execution will fail. Moreover, `create_primary_store` is currently permissionless.

>_ move/sources/registry_service.move RUST

```rust
/// Adds a wallet to an investor. Only callable by issuer or transfer agent or above.
public entry fun add_wallet(
    authorizer: &signer, investor_id: String, wallet_addr: address
) acquires InvestorInfo {
   [...]
    primary_fungible_store::create_primary_store(wallet_addr, asset);
    event::emit(
        DSRegistryServiceWalletAdded {
            wallet: wallet_addr,
            investorId: investor_id,
            sender: signer::address_of(authorizer)
        }
    );
}
```

Additionally, there is an incorrect assertion in `add_wallet_by_investor`, which verifies if `wallet_addr` is a special wallet, unintentionally allowing only special wallets to be added. This behavior may expose the system to risks. The intended functionality, however, is to prevent the registration of special wallets. Therefore, the assertion should check that `wallet_addr` is not a special wallet. A similar assertion should also be added to `add_wallet` to prevent the registration of special wallets.

>_ move/sources/registry_service.move RUST

```rust
/// Allows an investor to add a wallet to their own account.
public entry fun add_wallet_by_investor(
    authorizer: &signer, wallet_addr: address
) acquires InvestorInfo, WalletTypesRegistry {
    assert!(is_special_wallet(wallet_addr), EWALLET_SPECIAL);
    [...]
}
```

© 2024 Otter Audits LLC. All Rights Reserved.

Remediation

Update `add_wallet` and `add_wallet_by_investor` to call `ensure_primary_store_exists` instead of `create_primary_store`, to ensure that if a primary store already exists for the given address, `create_primary_store` will not be called for that address. Also, modify the assertion in `add_wallet_by_investor` to ensure that the `wallet_addr` is not a special wallet, and add a similar assertion in `add_wallet`.

Patch

Resolved in 06538b8 and 8932cf1.

© 2024 Otter Audits LLC. All Rights Reserved.

Abort Due to Underflow in Difference Calculation HIGH OS-ASC-ADV-09

Description

In `get_compliance_transferable_tokens_deposit` and `get_compliance_transferable_tokens` within `compliance_service`, `difference` is calculated as `time - lock_time` without first checking if `time` is greater than or equal to `lock_time`. If `lock_time` exceeds `time`, the subtraction operation (`time - lock_time`) will result in an underflow, causing the program to abort.

>_ *move/sources/compliance_service.move* RUST

```rust
fun get_compliance_transferable_tokens_deposit(
    who: address,
    time: u64,
    lock_time: u64,
    from_balance: u64
): u64 acquires ComplianceData {
    assert!(time > 0, EINVALID_TIME);
    [...]
    let total_tokens_locked = 0;
    for (i in 0..investor_issuances_count) {
        let issuances_timestamps = &compliance_data.issuances_timestamps;
        let issuances_timestamps_inner =
            smart_table::borrow(issuances_timestamps, investor);
        let issuance_time =
            *smart_table::borrow_with_default(issuances_timestamps_inner, i, &0);
        let difference = time - lock_time;
        if (lock_time > time || issuance_time > (difference as u256)) {
            let issuances_values = &mut compliance_data.issuances_values;
            let issuances_values_inner =
                smart_table::borrow_mut(issuances_values, investor);
            total_tokens_locked = total_tokens_locked
                + *smart_table::borrow_mut_with_default(issuances_values_inner, i, 0);
        }
    };
    [...]
}
```

Remediation

Check if `time >= lock_time` before performing the subtraction.

Patch

Resolved in 9f5d9b2.

© 2024 Otter Audits LLC. All Rights Reserved.

Wallet Balance Misverification MEDIUM OS-ASC-ADV-10

Description

`ds_token::check_wallets_for_list` checks the total token balance for an investor instead of the balance in each individual wallet. This implies that even if a wallet holds zero tokens, it will still be added to the active wallet list if the investor's total balance is non-zero. This discrepancy may allow an investor to create a large number of empty wallets that are added to the wallet tracking structures (`wallet_indexes` and `wallet_list`). Thus, an investor may create numerous dead wallets (wallets with a zero token balance), initiating a token transfer to each of these wallets with a value of zero.

>_ *move/sources/ds_token.move* RUST

```rust
/// Check wallets for list.
/// This function is called after every transfer, burn, mint, and seize operation.
fun check_wallets_for_list(from: address, to: address) acquires WalletData {
    let from_balance =
        registry_service::investor_wallet_balance_total(
            registry_service::get_investor(from)
        );
    let to_balance =
        registry_service::investor_wallet_balance_total(
            registry_service::get_investor(to)
        );
    if (from_balance == 0) {
        remove_wallet_from_list(from)
    };
    if (to_balance > 0) {
        add_wallet_to_list(to)
    };
}
```

Consequently, the function will add these wallets to `wallet_list` and `wallet_indexes`. With a large number of zero-balance wallets in `wallet_list` and `wallet_indexes`, the transfer functionality for some wallets may result in a hash-denial-of service scenario. However, executing this attack is costly, and there is no direct monetary reward to benefit from. Furthermore, it is a regulated environment where all wallets have to undergo a KYC procedure.

Additionally, `check_wallets_for_list` is called after each `transfer` and `transfer_from` operation. This function maintains a registry of active wallets, updating it to add wallets with positive balances and remove those with zero balances. However, this process does not account for transfers processed through `primary_fungible_store::deposit` and `primary_fungible_store::withdraw` flows, implying that certain wallet balance updates bypass `check_wallets_for_list`.

© 2024 Otter Audits LLC. All Rights Reserved.

Remediation

Verify the balance of each individual wallet in `check_wallets_for_list`. Also, move `check_wallets_for_list` to the end of the deposit to ensure it is called consistently, updating the wallet registry accurately after every deposit action regardless of the transfer method.

Patch

Resolved in 438f61f.

© 2024 Otter Audits LLC. All Rights Reserved.

Missing Bound Check on Lock Removal MEDIUM OS-ASC-ADV-11

Description

When a lock is removed, it is not actually deleted from the `SmartTable` storing lock records. This renders the data accessible in the system even after it is supposedly removed. Since locks are not fully removed from the `SmartTable`, view functions may show locks that should have been deleted. Also, the `lock_index` is not validated to ensure it is within the bounds of the investor's lock count. Thus, the `lock_index` values may be out of bounds, potentially attempting to delete nonexistent records. As a result, the same lock may be removed multiple times repeatedly, each time decreasing the lock count.

Remediation

Check that `lock_index <= total_investor_count` in both the view functions and the removal function.

Patch

Resolved in 06538b8.

© 2024 Otter Audits LLC. All Rights Reserved.

Failure to Clear Investor Attributes After Removal MEDIUM OS-ASC-ADV-12

Description

In `registry_service::remove_investor`, when an investor is removed, only their main record in the investors table is deleted. Any associated data, such as compliance attributes, remains in the attributes table. If a new investor is later registered with the same ID as the removed investor, the system will inadvertently link the new investor to the old attributes, potentially allowing unintended access to privileges based on deleted investor's attributes.

>_ *move/sources/registry_service.move* RUST

```rust
public entry fun remove_investor(authorizer: &signer, id: String) acquires InvestorInfo {
    [...]
    let investors = &mut investor_info_mut().investors;
    smart_table::remove(investors, id);
    event::emit(
        DSRegistryServiceInvestorRemoved {
            investorId: id,
            sender: signer::address_of(authorizer)
        }
    );
}
```

Remediation

Remove the investor's attributes from the attributes table when the investor record is deleted.

Patch

Resolved in 06538b8.

© 2024 Otter Audits LLC. All Rights Reserved.

Investor Limit Calculation Error MEDIUM OS-ASC-ADV-13

Description

`max_us_investors_percentage` is not considered when calculating the U.S. investors' limit. The U.S. investors' limit is intended to restrict the percentage of U.S. investors relative to the total number of investors. Not incorporating the `max_us_investors_percentage` implies that the actual cap on U.S. investors may exceed the intended compliance threshold.

Remediation

Calculate the limit as shown below:

$$\text{Actual Limit} = \min\left(\text{us_investors_limit}, \frac{\text{max_us_investors_percentage} \times \text{total_investors}}{100}\right)$$

Apply this logic if both `us_investors_limit` and `max_us_investors_percentage` are non-zero.

Patch

Resolved in #61.

© 2024 Otter Audits LLC. All Rights Reserved.

Incongruities in Role Management Checks LOW OS-ASC-ADV-14

Description

There are several inconsistencies with the role assignment in the system. In `registry_service` for `register_investor` (shown below) and `add_wallet`, access is restricted to only specific roles, limiting it to issuers, transfer agents, or higher authority roles. However, the requirement is that these functions should be callable by any role, implying that the `Exchange` role (represented by role ID 4) should also be able to call these functions. Ensure that the `Exchange` role is also allowed to call the above two functions.

>_ *move/sources/registry_service.move* RUST

```rust
public entry fun register_investor(
    authorizer: &signer, id: String, collision_hash: String
) acquires InvestorInfo {
    trust_service::assert_only_issuer_or_transfer_agent_or_above(authorizer);
    assert!(!is_investor(id), EINVESTOR_ALREADY_EXISTS);
    [...]
}
```

Remediation

Address the above discrepancies in role management.

Patch

Resolved in 06538b8.

© 2024 Otter Audits LLC. All Rights Reserved.

Utilization of Empty Investor ID LOW OS-ASC-ADV-15

Description

In `registry_service::register_investor`, an empty string as an ID may be treated differently from other valid IDs. An empty string ID has a special meaning in the contract and will break many parts of the logic if an investor registers with such an ID. An example is that `is_wallet` will return true for any wallet, as if the investor does not exist, `get_investor` will return `""`. This will create confusion in the contract's logic, where functions that check for investor existence may incorrectly identify an empty ID as a valid investor.

>_ *move/sources/registry_service.move* RUST

```rust
/// Registers a new investor with the given ID and collision hash. Only callable by issuer or
    ↪ transfer agent or above.
public entry fun register_investor(
    authorizer: &signer, id: String, collision_hash: String
) acquires InvestorInfo {
    trust_service::assert_only_issuer_or_transfer_agent_or_above(authorizer);
    assert!(!is_investor(id), EINVESTOR_ALREADY_EXISTS);
    let investor = Investor {
        id,
        collision_hash,
        creator: signer::address_of(authorizer),
        last_updated_by: signer::address_of(authorizer),
        country: string::utf8(b""),
        wallet_count: 0,
        wallets: vector::empty()
    };
    [...]
}
```

Remediation

Add an assertion to ensure that the ID is not an empty string (`id != ""`).

Patch

Resolved in 06538b8.

© 2024 Otter Audits LLC. All Rights Reserved.

Missing Last Modifier Update LOW OS-ASC-ADV-16

Description

`registry_service::set_attribute` does not currently update the `last_updated_by` field within the `InvestorInfo` structure for an investor when setting or updating an attribute. This field should record the last entity that modified any attribute for that investor. Since `last_updated_by` remains unchanged, it becomes impossible to trace which `authorizer` was responsible for the most recent modification to an investor's attributes.

Remediation

Modify `set_attribute` to ensure that the `last_updated_by` field in the `InvestorInfo` structure is set to the `authorizer`'s address whenever an attribute is added or updated.

Patch

Resolved in 06538b8.

© 2024 Otter Audits LLC. All Rights Reserved.

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-ASC-SUG-00	`ds_token::approve` allows an investor to grant spending approval to another address, but lacks a way to completely remove this approval.
OS-ASC-SUG-01	There are several instances where proper validation is not done, resulting in potential security issues.
OS-ASC-SUG-02	Additional safety checks may be incorporated within the codebase to make it more robust and secure.
OS-ASC-SUG-03	Recommendation for modifying the codebase for improved functionality, efficiency, and maintainability.

© 2024 Otter Audits LLC. All Rights Reserved.

Missing Approval Removal Function OS-ASC-SUG-00

Description

`ds_token::approve` allows an investor to grant spending permissions to another address (`spender`). However, there is no direct way to withdraw or remove this approval from the allowances list. Instead, the only option is to reset the approval amount to zero.

Remediation

Add a function to revoke any given spender's permission by deleting their entry from the `allowances` list altogether.

Patch

Resolved in 06538b8.

© 2024 Otter Audits LLC. All Rights Reserved.

Missing Validation Logic OS-ASC-SUG-01

Description

1. In the current implementation, `init_module` functions do not verify that the deployer's address matches the predefined address stored in `ds_protocol`. This is especially necessary since key helper functions such as `dstoken_address` rely on this specific address.
2. In `token_issuer::issue_tokens`, instead of merely verifying that `get_investor(to)` is not empty, asserting that `get_investor(to) == id` will provide additional accuracy by ensuring that the recipient's wallet (`to`) is actually associated with the specified investor (`id`).

   >_ *move/sources/token_issuer.move* RUST

   ```rust
   public entry fun issue_tokens(
       [...]
   ) {
       [...]
       if (registry_service::is_wallet(to)) {
           assert!(registry_service::get_investor(to) != string::utf8(b""), 0);
       }[...]
   }
   ```

3. `registry_service::add_wallet` does not include the `assert_new_wallet` check to ensure that the wallet address does not already exist for the specified investor before adding it.
4. Currently, there is no check in `create_lock_for_investor` to verify if the investor ID is valid and actually exists.

Remediation

1. Verify that the deployer's address aligns with the stored one.
2. Enforce a strict association between the wallet address (`to`) and the specified investor ID (`id`) to ensure that tokens are issued to the correct investor.
3. Verify that a wallet does not already exist for an investor via `assert_new_wallet`.
4. Ensure `create_lock_for_investor` checks if the investor exists before creating the lock.

Patch

1. Issue #1 resolved in 06538b8.
2. Issue #2 resolved in 06538b8.
3. Issue #3 resolved in 06538b8.

© 2024 Otter Audits LLC. All Rights Reserved.

Additional Safety Checks

OS-ASC-SUG-02

Description

1. `registry_service::set_attribute` should ensure that the `attribute_value` corresponds to a set of acceptable values defined for each `attribute_id`, to prevent assigning values that do not align with the specific attribute's intended purpose. Additionally, confirm that `attribute_expiration` is within the allowed range.
2. The current implementation of `compliance_service::validate_issuance_time` only checks if back-dating is allowed. It does not verify whether the provided `issuance_time` is less than or equal to the current timestamp.

   >_ *move/sources/compliance_service.move* RUST

   ```rust
   fun validate_issuance_time(issuance_time: u256): u256 acquires Miscellaneous {
       if (!get_disallow_back_dating()) {
           return issuance_time
       };
       return (timestamp::now_seconds() as u256)
   }
   ```

3. There is no address validation in `ds_token::check_wallets_for_list` to ensure it does not perform any list operations for the zero address (`@0x0`).

Remediation

1. Include the additional validation checks on both the `attribute_value` and `attribute_expiration` fields, as mentioned above.
2. Add a check to ensure that the issuance time is not in the future.
3. Verify that `check_wallets_for_list` does not operate on a zero address.

© 2024 Otter Audits LLC. All Rights Reserved.

Code Refactoring OS-ASC-SUG-03

Description

1. In `register_service::add_wallet_by_investor`, both the `creator` and `last_updated_by` fields of the `Wallet` struct are set to `wallet_addr`, which is the address of the added wallet. This implies that the wallet is attributed to itself as its creator and last updater. Change the `creator` and `last_updated_by` fields, as well as the `sender` field in the emitted event, to utilize `signer::address_of(authorizer)` instead of `wallet_addr`.

>_ *move/sources/registry_service.move* RUST

```rust
public entry fun add_wallet_by_investor(
    authorizer: &signer, wallet_addr: address
) acquires InvestorInfo, WalletTypesRegistry {
    [...]
    smart_table::add(
        investor_wallets,
        wallet_addr,
        Wallet { owner, creator: wallet_addr, last_updated_by: wallet_addr }
    );
    [...]
    event::emit(
        DSRegistryServiceWalletAdded {
            wallet: wallet_addr,
            investorId: owner,
            sender: wallet_addr
        }
    );
}
```

2. Abort `lock_manager::remove_lock_record` in cases where the investor or the specified `lock_index` does not exist to improve code reliability and user experience. Silent failures may result in ambiguity, as the caller may not be aware that the intended operation did not succeed.
3. The `FORBIDDEN` constant is defined with different values in different modules. It is declared as 3 in `registry_service` and 4 in `compliance_service`. Ensure that the value is the same in both modules for consistency.
4. Returning status and error codes in `pre_issuance_check` and `validate_deposit` is unnecessary, as the only possible code is `VALID`. For any other code, the function will abort.
5. To ensure proper removal of an investor, remove the investor's wallets from `investors_wallets`.

© 2024 Otter Audits LLC. All Rights Reserved.

Remediation

Implement the above-mentioned suggestions.

Patch

1. Issue #1 resolved in 06538b8.
2. Issue #2 resolved in 06538b8.

© 2024 Otter Audits LLC. All Rights Reserved.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2024 Otter Audits LLC. All Rights Reserved.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2024 Otter Audits LLC. All Rights Reserved.